

UNIVERSITY PARTNER



Artificial Intelligence and Machine Learning (6CS012)

Title - Scene Classification using Convolutional Neural
Networks

Team:

Name	University ID
Rajiv Khattri	2408207
Binaya Bohara	2418248
Unik Ghalan	2409043

1. Research Question

This project investigates the following central research question:

Can a Convolutional Neural Network effectively classify natural outdoor scenes into six categories, and does transfer learning with a pre-trained model outperform a CNN trained entirely from scratch?

This question is addressed through two complementary tasks: (a) designing and training CNN architectures from scratch with increasing complexity, and (b) fine-tuning a pre-trained ImageNet model using transfer learning. The results of both approaches are compared systematically using standard evaluation metrics.

2. Title

Scene Classification Using Convolutional Neural Networks and Transfer Learning

3. Dataset Description

3.1 Source and Overview

The dataset selected for this project is the Intel Image Classification Dataset, originally published and made publicly available on Kaggle. The dataset was curated by Intel as a scene classification benchmark and has since become a widely used resource in academic and applied deep learning research. Prior approval was obtained from the module instructor before finalizing this dataset selection, in accordance with the assignment guidelines.

Dataset Link -

https://drive.google.com/drive/folders/1FvrV5yaJr7LyuvqT_Ie1U2STYhQILVxN?usp=sharing

The dataset consists of natural outdoor scene images organized into six distinct semantic categories: Buildings, Forest, Glacier, Mountain, Sea, and Street. These categories

represent visually diverse real-world environments, making the dataset suitable for evaluating a model's capacity to learn discriminative visual features.

3.2 Dataset Statistics

Class	Training Images	Test Images	Total	Total Corrupted Images
Buildings	2,191	437	2,628	7
Forest	2,271	474	2,745	7
Glacier	2,404	553	2,957	7
Mountain	2,512	525	3,037	7
Sea	2,274	510	2,784	7
Street	2,382	501	2,883	7
Total	14,034	3,000	17,034	42

Table 1: Class-wise distribution of images across training and test splits.

3.3 Image Characteristics

All images in the dataset are stored in JPEG format and are approximately 150×150 pixels in resolution with three RGB color channels. The images were collected under diverse real-world conditions, including varying lighting, weather, and camera angles, which increases the difficulty and practical relevance of the classification task. The dataset arrives in a pre-structured directory format with separate train and test folders, each containing one sub-folder per class.

During initial inspection, a small number of corrupt files (approximately 10 bytes in size) were identified within the training directory. These will be detected and excluded programmatically during the data loading stage to prevent training errors.

4. Data Analysis

4.1 Class Distribution

A preliminary analysis of the class distribution reveals that the dataset is nearly balanced. The largest class (Mountain) contains 2,512 training images, while the smallest (Buildings) contains 2,191 - a difference of approximately 15%. This level of imbalance is considered acceptable for CNN training without requiring class weighting or oversampling techniques. Also there are 42 corrupted images in total and 7 corrupted images in each class. Standard accuracy is therefore a valid primary evaluation metric for this dataset.

4.2 Visual Observations

Visual inspection of sample images from each class reveals the following characteristics:

- **Buildings:** Urban architectural scenes featuring man-made structures, roads, and city environments. Generally high contrast with sharp edges.
- **Forest:** Dense vegetation with varied shades of green. High texture complexity due to foliage patterns.
- **Glacier:** Ice and snow-covered terrain with predominantly white, blue, and grey tones. Visually similar to Mountain class.
- **Mountain:** Rocky terrain, often with snow caps. Overlapping color palette with Glacier class, which is expected to cause inter-class confusion.
- **Sea:** Open water bodies with blue and teal tones. Can visually overlap with Glacier class in certain lighting conditions.
- **Street:** Urban road scenes with vehicles, pedestrians, and infrastructure. Visually distinct from natural scene classes.

4.3 Anticipated Challenges

Based on the analysis, the following challenges are anticipated during model training and evaluation:

- **Inter-class visual similarity:** Glacier and Mountain share overlapping color distributions and textures. Similarly, Sea and Glacier images can appear visually

close in certain lighting conditions. These pairs are likely to generate the highest number of misclassifications.

- Overfitting risk: Deeper CNN architectures with a large number of parameters may overfit to the 14,034 training images. This will be addressed through Dropout, Batch Normalization, and data augmentation.
- Input size mismatch for transfer learning: Pre-trained models such as VGG16 and ResNet50 require 224×224 pixel inputs, whereas the native image resolution is 150×150. Images will be resized using Keras ImageDataGenerator for Part B.
- Computational constraints: Training deeper architectures requires significant compute resources. Google Colab with GPU acceleration will be used throughout the project.

5. Planned Methodology

5.1 Data Preparation

All images will be preprocessed using the Keras ImageDataGenerator class. For the training set, preprocessing will include pixel normalization (rescaling values from [0, 255] to [0, 1]) and data augmentation techniques including horizontal flipping, rotation up to 20 degrees, zoom up to 20%, and width and height shifts. Augmentation is applied exclusively to the training set to prevent data leakage into validation and test evaluation. A validation split of 20% will be carved from the training set, yielding approximately 11,200 training and 2,800 validation images. For Part B, images will additionally be resized to 224×224 pixels to match the input requirements of pre-trained ImageNet models.

5.2 Part A - CNN from Scratch

Two CNN models will be designed and trained from scratch:

- Baseline Model: Three convolutional layers each followed by a max-pooling layer, with filter counts of 32, 64, and 128 respectively. Three fully connected dense layers and a 6-class softmax output layer. ReLU activation is used throughout.

- **Deeper Model:** At least double the number of convolutional layers compared to the baseline, with the addition of Batch Normalization after each convolutional block and Dropout (rate 0.5) before the dense layers. This model is designed to investigate whether increased depth and regularization improves generalization.

Both models will be trained using the Adam optimizer initially, with a comparative experiment using SGD on the deeper model. An ablation study will be conducted by removing one regularization component (e.g., Dropout) to quantify its contribution to performance.

5.3 Part B - Transfer Learning

VGG16, pre-trained on ImageNet, will be used as the backbone for transfer learning. The original 1000-class classification head will be removed and replaced with a custom head consisting of a Flatten layer, a Dense layer with 256 neurons and ReLU activation, a Dropout layer (rate 0.5), and a final Dense layer with 6 neurons and softmax activation. The convolutional base will initially be frozen for feature extraction, followed by a fine-tuning stage in which the final convolutional block is unfrozen and trained at a lower learning rate.

5.4 Evaluation

All models will be evaluated using accuracy, precision, recall, F1-score (via classification report), and confusion matrix visualization. Training and validation loss curves will be plotted for each model. Results across all models will be compared in a summary table within the final report.

6. Tools and Technologies

Tool / Library	Purpose
Python 3.x	Primary programming language
TensorFlow / Keras	CNN model design, training, and transfer learning
NumPy	Numerical computation and array manipulation

Matplotlib	Plotting loss/accuracy curves and visualizations
Scikit-learn	Confusion matrix, classification report, metrics
Google Colab (GPU)	Cloud-based training with hardware acceleration

Table 2: Tools and technologies to be used in the project.